

Mobile Cellular Baseband & 5G Network Boundary Forensics

Complete Documentation & User Guide

Python + Streamlit + ADB | Android Forensics Platform

Real-World Android Baseband & 5G NR Forensic Analysis Platform
Built with Python & Streamlit ? Works with any Android device via ADB

Table of Contents

1. Overview
2. What It Does
3. System Architecture
4. Prerequisites
5. Installation
6. Quick Start
7. Connecting Your Android Device
8. Using the Dashboard
9. Understanding the Results
10. Report Export
11. Project Structure
12. Module Reference
13. Troubleshooting
14. Docker Deployment
15. Disclaimer
16. License

? Overview

Mobile Cellular Baseband & 5G Network Boundary Forensics is a comprehensive forensic analysis tool that connects to Android devices to extract, parse, and analyze cellular baseband data, 5G NR network parameters, SIM card artifacts, and cell tower information.

It works with any Android device -- no root required. Connect via USB or WiFi, click one button, and get a full forensic report in JSON, HTML, and PDF formats.

Use Cases

- Digital Forensics -- Extract evidence from Android cellular subsystems
- Security Research -- Audit baseband security posture and detect anomalies
- Network Analysis -- Map 5G NR deployments, carrier aggregation, network slicing
- Penetration Testing -- Detect IMSI catcher indicators, 2G downgrade attacks
- Mobile Network Engineering -- Verify 5G SA/NSA deployment, signal measurements

? What It Does

1. Baseband Analysis

- Identifies chipset vendor (Qualcomm, Samsung, MediaTek, HiSilicon, Unisoc, Intel)
- Extracts baseband firmware version, RIL version, Radio HAL version
- Detects engineering/debug builds and test-key signatures
- Enumerates supported cellular technologies (NR, LTE, WCDMA, GSM)
- Checks VoLTE, VoWiFi, 5G NSA, and 5G SA capabilities

2. Radio Layer Forensics

- Parses Android's Radio Interface Layer (RIL) events from logcat -b radio
- Extracts cell tower history with timestamps, MCC/MNC, Cell ID, PCI, TAC
- Builds signal strength timeline (RSRP, RSRQ, RSSI, SINR)
- Detects cell reselection patterns, radio link failures, registration changes

3. 5G NR Network Analysis

- Detects 5G NSA (Non-Standalone) vs SA (Standalone) mode
- Identifies NR bands, NR-ARFCN, subcarrier spacing (SCS), channel bandwidth
- Analyzes carrier aggregation (CA) and EN-DC combinations
- Detects network slicing indicators (SST, SD, NSSAI)
- Estimates theoretical throughput based on 3GPP TS 38.306 parameters

4. SIM/USIM Forensics

- Extracts ICCID and IMSI with 200+ country/operator lookups
- Identifies SIM type (SIM, USIM, eSIM, CSIM)
- Detects roaming status
- Extracts MSISDN (phone number), IMEI, card issuer identification
- Reads PIN/PUK retry counts, GID1/GID2, SPN

5. Cell Tower Mapping

- Maps all detected cell towers with technology, band, signal strength
- Estimates distance to each tower using free-space path loss
- Classifies signal quality (Excellent ? No Signal)
- Generates GeoJSON for external mapping tools

6. Security Audit

- Scores device risk from 0-100 across 10+ checks
- Reports root access, bootloader unlock, SELinux status
- Verifies DM-Verity and encryption state
- Calculates security patch age
- Identifies dangerous services and exposed diagnostic interfaces

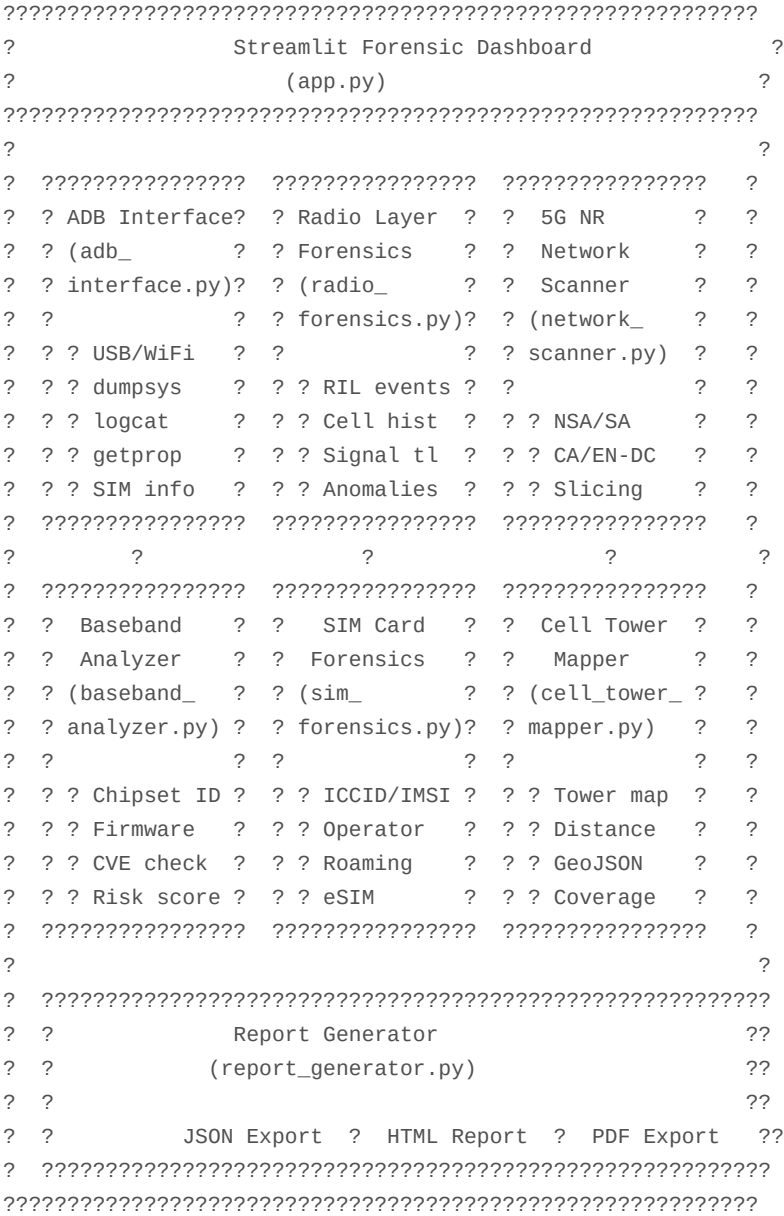
7. Anomaly Detection

- IMSI attach/detach frequency monitoring
- Cell reselection/handover pattern analysis
- Radio link failure detection
- Null/weak cipher detection
- 2G fallback detection (potential downgrade attack)
- IMSI catcher/stingray indicators
- 5G BWP switching anomalies

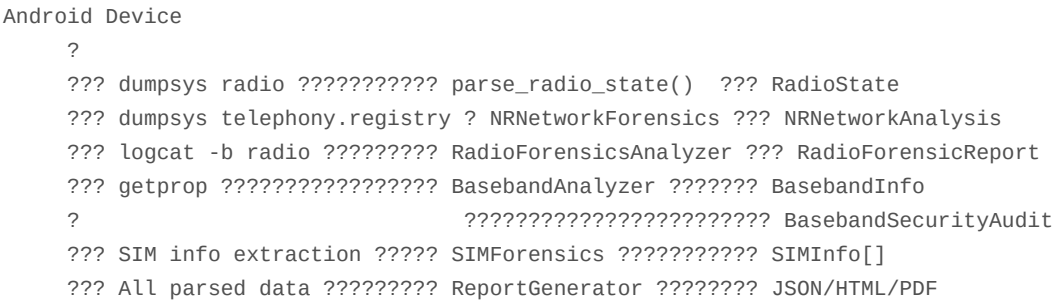
8. Report Generation

- JSON -- Machine-readable, complete forensic data
- HTML -- Self-contained, dark-themed, shareable report
- PDF -- Formatted document with all 7 analysis sections

System Architecture



Data Flow



Prerequisites

Hardware

Requirement | Details

Computer | Windows 10/11, macOS 10.15+, or Linux

Android Device | Any Android phone/tablet (Android 5.0+)

USB Cable | Data-capable USB cable (for USB connection)

Software

Requirement | Version | How to Install

Python | 3.10 or newer | python.org/downloads

ADB (Android Platform Tools) | Latest | developer.android.com/studio/releases/platform-tools

Python Packages

```
streamlit>=1.30.0    # Web dashboard
pandas>=2.0.0        # Data handling
numpy>=1.24.0        # Numerical operations
plotly>=5.15.0       # Interactive charts
fpdf2>=2.7.0         # PDF generation
```

Installation

Step 1: Clone the Repository

```
git clone https://github.com/graks/Mobile-Cellular-Baseband-5G-Network-Boundary-Forensics.git
cd Mobile-Cellular-Baseband-5G-Network-Boundary-Forensics
```

Step 2: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 3: Install ADB

Windows:

Download and extract

```
Invoke-WebRequest https://dl.google.com/android/repository/platform-tools-latest-windows.zip -OutFile platform-tools.zip
```

```
Expand-Archive platform-tools.zip -DestinationPath C:\platform-tools
```

macOS:

```
brew install android-platform-tools
```

Linux:

```
sudo apt install adb
```

Step 4: Verify ADB

```
adb version
```

Should show: Android Debug Bridge version 1.0.41

Quick Start

1. Launch the Dashboard

```
streamlit run app.py
```

2. Open Your Browser

Navigate to <http://localhost:8501>

3. Prepare Your Android Device

- Go to Settings ? About Phone
- Tap Build Number 7 times until "You are now a developer!"
- Go to Settings ? Developer Options
- Enable USB Debugging

4. Connect and Analyze

- Plug your device into your computer via USB
- Accept the "Allow USB debugging?" prompt on your phone
- In the dashboard sidebar, paste your ADB path (e.g., C:\platform-tools\adb.exe)
- Click Scan for Devices
- Select your device from the dropdown
- Click Run Full Analysis

The analysis takes 30-60 seconds. Results appear in 7 interactive tabs.

Connecting Your Android Device

Method 1: USB Connection

1. Enable USB Debugging (see above)
2. Plug phone into computer via USB cable
3. Accept the RSA fingerprint prompt on the phone
4. In the dashboard: select USB ? Scan for Devices

Method 2: Wireless (WiFi) Connection

First-time setup (requires USB once):

```
adb tcpip 5555
```

Then unplug the USB cable.

1. Find your phone's IP address: Settings ? About Phone ? Status ? IP Address
2. In the dashboard: select Wireless (TCP/IP)
3. Enter your phone's IP and port 5555
4. Click Connect via WiFi
5. Click Scan for Devices

> Note: Phone and computer must be on the same WiFi network.

Method 3: Wireless Debugging (Android 11+)

No USB cable needed at all:

1. Settings ? Developer Options ? Wireless debugging ? ON
2. Tap Pair device with pairing code
3. Run in terminal:

```
adb pair :
```

```
adb connect :
```

? Using the Dashboard

Sidebar

Element | Purpose

ADB path | Paste the full path to adb.exe (e.g., C:\platform-tools\adb.exe)

Connection method | Switch between USB and Wireless (TCP/IP)

Device IP / Port | For WiFi connections (port defaults to 5555)

Scan for Devices | Refresh the device list

Device selector | Choose which device to analyze

Run Full Analysis | Start the forensic analysis pipeline

Tabs

Tab | Content

Radio State | Current network type, signal strength gauge, cell tower details, raw radio dumps

5G NR | NSA/SA mode, NR bands, serving cell parameters, throughput estimation, carrier aggregation, network slicing

Security | Risk score gauge, security posture details, baseband processor information

SIM Card | ICCID, IMSI, operator, country, roaming status, MSISDN, IMEI

Towers | All detected cell towers, technology/signal distribution charts

Events | Parsed radio logcat events, category/type charts, signal timeline

Report | Anomaly summary, download buttons for JSON/HTML/PDF reports

Understanding the Results

Signal Quality Classification

RSRP Range | Quality

? -80 dBm | Excellent

-80 to -90 dBm | Good

-90 to -100 dBm | Fair

-100 to -110 dBm | Poor

-110 to -120 dBm | Very Poor

< -120 dBm | No Signal

5G Modes

Mode | Meaning

5G NR (SA) | Standalone -- directly connected to a 5G core network

5G NR (NSA) | Non-Standalone -- 5G radio with LTE core (EN-DC)

4G LTE | Connected to LTE only

Risk Levels

Score | Level | Action

0-9 | MINIMAL | Device is well-secured

10-29 | LOW | Minor concerns

30-49 | MEDIUM | Some issues to investigate

50-69 | HIGH | Significant security gaps

70-100 | CRITICAL | Major security compromise

Common Anomalies

2G fallback detected | Phone dropped from 4G/5G to 2G -- possible downgrade attack or coverage gap

High IMSI attach frequency | Possible tracking or network instability

Null/weak cipher detected | Radio connection using no/weak encryption -- security risk

IMSI catcher indicators | Potential stingray/IMSI catcher detected

High cell reselection count | Rapid mobility or coverage issues

Radio link failures | Multiple connection drops detected

Report Export

JSON Report

Complete machine-readable forensic data. Use for:

- Importing into other analysis tools
- Automated processing pipelines
- Evidence preservation with cryptographic hashing

HTML Report

Self-contained, dark-themed formatted report. Use for:

- Sharing findings with non-technical stakeholders
- Printing or PDF conversion from browser
- Presentation-ready documentation

PDF Report

Seven-section formatted document including:

1. Security Posture
2. Baseband Information
3. Current Radio State
4. SIM Card Analysis
5. 5G NR Analysis
6. Detected Anomalies
7. Cell Tower Map (with formatted table)

All downloadable from the Report tab with one click.

Project Structure

Mobile-Cellular-Baseband-5G-Network-Boundary-Forensics/

```
?
??? app.py                # Streamlit dashboard (main entry point)
??? requirements.txt      # Python package dependencies
??? Dockerfile           # Docker container definition
??? run.sh               # Quick launch shell script
?
??? modules/
    ??? __init__.py       # Package marker
    ?
    ??? adb_interface.py  # ADB Communication Module
    ?   ??? ADBInterface  # Manages all ADB commands
    ?   ??? DeviceInfo    # Device identification dataclass
    ?   ??? RadioState    # Radio state dataclass
    ?   ??? parse_radio_state() # Parses dumphsys into RadioState
    ?
```

```

??? radio_forensics.py          # Radio Layer Forensics Module
?   ??? RadioForensicsAnalyzer  # Parses RIL events, detects anomalies
?   ??? RadioForensicReport     # Complete forensic report dataclass
?
??? baseband_analyzer.py        # Baseband Analysis Module
?   ??? BasebandAnalyzer        # Chipset ID, firmware analysis
?   ??? BasebandInfo            # Baseband information dataclass
?   ??? BasebandSecurityAudit   # Security audit dataclass
?
??? network_scanner.py          # 5G NR Network Scanner Module
?   ??? NRNetworkForensics      # NR cell analysis
?   ??? NRCell                  # NR cell dataclass
?   ??? NRNetworkAnalysis       # Network analysis dataclass
?
??? sim_forensics.py            # SIM/USIM Forensics Module
?   ??? SIMForensics            # ICCID/IMSI parsing
?   ??? SIMInfo                 # SIM information dataclass
?
??? cell_tower_mapper.py        # Cell Tower Mapping Module
?   ??? CellTowerMapper         # Tower mapping and GeoJSON
?   ??? TowerRecord             # Individual tower dataclass
?   ??? TowerMap                # Complete tower map dataclass
?
??? report_generator.py         # Report Export Module
    ??? ReportGenerator         # JSON/HTML/PDF generation
    ??? ForensicReport          # Unified forensic report dataclass

```

? Module Reference

adb_interface.py

Method	Returns	Description
check_adb_available()	bool	Verify ADB is installed
list_devices()	List[DeviceInfo]	List connected devices with details
connect_wireless(ip, port)	(bool, str)	Connect to device over WiFi
disconnect_wireless(ip, port)	str	Disconnect WiFi connection
set_device(serial)	--	Set target device
get_radio_dump()	str	Get radio state (3 fallback strategies)
get_telephony_dump()	str	Get telephony registry
get_baseband_logs(lines)	str	Get radio logcat (3 buffer fallback)
get_modem_logs(lines)	str	Get modem logcat
get_sim_info()	Dict	Extract SIM card information
check_root()	bool	Check if device is rooted
get_radio_hal_version()	str	Get Radio HAL version

radio_forensics.py

Method	Returns	Description
parse_logcat_radio_events(logcat)	List[Dict]	Parse logcat into structured events
extract_cell_history(events)	List[CellRecord]	Extract cell tower history
detect_anomalies(events, state)	List[str]	Detect forensic anomalies
build_signal_timeline(events)	List[Dict]	Build signal strength timeline
generate_forensic_report(...)	RadioForensicReport	Generate complete report

baseband_analyzer.py

Method | Returns | Description

identify_chipset(props, build_prop) | (vendor, proc, model) | Identify baseband chipset
analyze_baseband(...) | BasebandInfo | Comprehensive baseband analysis
audit_security(...) | BasebandSecurityAudit | Security audit with risk scoring
generate_baseband_fingerprint(...) | str | SHA-256 fingerprint

network_scanner.py

Method | Returns | Description

parse_nr_cell_info(dump) | List[NRCell] | Parse NR cell information
analyze_carrier_aggregation(...) | List[Dict] | Analyze CA/EN-DC combos
detect_network_slicing(...) | List[str] | Detect slicing indicators
estimate_throughput(cell) | Dict | Estimate 5G throughput
analyze_5g_network(...) | NRNetworkAnalysis | Full 5G NR analysis

sim_forensics.py

Method | Returns | Description

parse_iccid(iccid) | Dict | Parse ICCID structure
parse_imsi(imsi) | Dict | Parse IMSI into MCC/MNC/MSIN
analyze_sim(...) | List[SIMInfo] | Full SIM forensic analysis

cell_tower_mapper.py

Method | Returns | Description

classify_signal_quality(rsrp) | str | Classify signal quality
estimate_distance(dbm, tech, freq) | float | Estimate tower distance
map_towers(...) | TowerMap | Map all detected towers
generate_geojson(tower_map) | Dict | Generate GeoJSON

report_generator.py

Method | Returns | Description

generate_report_id() | str | Generate unique report ID
compile_report(...) | ForensicReport | Compile all analysis modules
to_json(report) | str | Export as JSON
to_html(report) | str | Export as HTML
to_pdf(report) | bytes | Export as PDF
export_report(report, format) | str/bytes | Export in specified format

Troubleshooting

"ADB not found"

Solution: Paste the full path to adb.exe in the sidebar's ADB path field.

Find your ADB location:

Windows: where adb

macOS/Linux: which adb

"No devices detected"

Check:

1. USB Debugging is enabled on the phone
2. You accepted the RSA fingerprint prompt on the phone
3. The USB cable supports data transfer (not charge-only)
4. Try `adb kill-server` then `adb start-server`

"0 dBm signal"

The parser couldn't extract signal from the dump format. This happens on some manufacturers. The tool tries 5 different extraction patterns. Check the Raw Radio Dump expander to see the actual format.

"Command timed out"

WiFi connections can be slow. Increase timeout or switch to USB.

"Connection refused" (WiFi)

Restart ADB in TCP mode:

```
adb usb
adb tcpip 5555
adb connect :5555
```

PDF errors

Make sure `fpdf2` is installed:

```
pip install fpdf2
```

? Docker Deployment

Build

```
docker build -t baseband-forensics .
```

Run

```
docker run -p 8501:8501 \
  -v /path/to/adb:/usr/bin/adb \
  --device /dev/bus/usb \
  baseband-forensics
```

> Note: USB device passthrough requires `--privileged` on some systems. For WiFi-only usage, no special flags needed.

!? Disclaimer

This tool is for authorized forensic analysis and security research purposes only.

- Always obtain proper authorization before analyzing any device
- Respect privacy laws and regulations in your jurisdiction
- The authors assume no liability for misuse of this software
- Use of this tool on devices you do not own or have explicit permission to analyze may be illegal

? License

MIT License -- see LICENSE file for details.

? Acknowledgments

- Android Open Source Project (AOSP) for Radio Interface Layer documentation
- 3GPP for NR specifications (TS 38.104, TS 38.306)
- Streamlit team for the excellent dashboard framework
- FPDF2 for PDF generation

Built with ?? for the digital forensics community